

1. Load and Inspect the Data

In [254...

```
import pandas as pd
import numpy as np

# Load dataset
df = pd.read_csv('customer.csv')

# Check structure, data types, and missing values
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 3900 entries, 0 to 3899
Data columns (total 18 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Customer ID                          3900 non-null   int64
1   Age                                   3900 non-null   int64
2   Gender                               3900 non-null   str
3   Item Purchased                       3900 non-null   str
4   Category                             3900 non-null   str
5   Purchase Amount (USD)                3900 non-null   int64
6   Location                             3900 non-null   str
7   Size                                 3900 non-null   str
8   Color                                3900 non-null   str
9   Season                               3900 non-null   str
10  Review Rating                        3863 non-null   float64
11  Subscription Status                  3900 non-null   str
12  Shipping Type                        3900 non-null   str
13  Discount Applied                     3900 non-null   str
14  Promo Code Used                      3900 non-null   str
15  Previous Purchases                   3900 non-null   int64
16  Payment Method                       3900 non-null   str
17  Frequency of Purchases                3900 non-null   str
dtypes: float64(1), int64(4), str(13)
memory usage: 835.9 KB
```

In [255...

```
df.head()
```

Out[255...

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	Color
0	1	55	Male	Blouse	Clothing	53	Kentucky	L	Gray
1	2	19	Male	Sweater	Clothing	64	Maine	L	Maroon
2	3	50	Male	Jeans	Clothing	73	Massachusetts	S	Maroon
3	4	21	Male	Sandals	Footwear	90	Rhode Island	M	Maroon
4	5	45	Male	Blouse	Clothing	49	Oregon	M	Turquoise

In [256...

```
df.describe(include='all')
```

Out[256...

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size
count	3900.000000	3900.000000	3900	3900	3900	3900.000000	3900	3900
unique	NaN	NaN	2	25	4	NaN	50	50
top	NaN	NaN	Male	Blouse	Clothing	NaN	Montana	NaN
freq	NaN	NaN	2652	171	1737	NaN	96	1737
mean	1950.500000	44.068462	NaN	NaN	NaN	59.764359	NaN	NaN
std	1125.977353	15.207589	NaN	NaN	NaN	23.685392	NaN	NaN
min	1.000000	18.000000	NaN	NaN	NaN	20.000000	NaN	NaN
25%	975.750000	31.000000	NaN	NaN	NaN	39.000000	NaN	NaN
50%	1950.500000	44.000000	NaN	NaN	NaN	60.000000	NaN	NaN
75%	2925.250000	57.000000	NaN	NaN	NaN	81.000000	NaN	NaN
max	3900.000000	70.000000	NaN	NaN	NaN	100.000000	NaN	NaN

In [257...

```
df.isnull().sum()
```

```
Out[257... Customer ID      0
           Age          0
           Gender       0
           Item Purchased 0
           Category     0
           Purchase Amount (USD) 0
           Location     0
           Size         0
           Color        0
           Season       0
           Review Rating 37
           Subscription Status 0
           Shipping Type 0
           Discount Applied 0
           Promo Code Used 0
           Previous Purchases 0
           Payment Method 0
           Frequency of Purchases 0
           dtype: int64
```

2. Handle Missing Values & Feature Engineering

```
In [258... # Impute missing Review Ratings with the median rating
df['Review Rating'] = df['Review Rating'].fillna(df['Review Rating'].median())
```

```
In [259... df
```

Out[259...

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	
0	1	55	Male	Blouse	Clothing	53	Kentucky	L	
1	2	19	Male	Sweater	Clothing	64	Maine	L	Ma
2	3	50	Male	Jeans	Clothing	73	Massachusetts	S	Ma
3	4	21	Male	Sandals	Footwear	90	Rhode Island	M	Ma
4	5	45	Male	Blouse	Clothing	49	Oregon	M	Turq
...	...	...	...	...	...	...	...	...	
3895	3896	40	Female	Hoodie	Clothing	28	Virginia	L	Turq
3896	3897	52	Female	Backpack	Accessories	49	Iowa	L	V
3897	3898	46	Female	Belt	Accessories	33	New Jersey	L	C
3898	3899	44	Female	Shoes	Footwear	77	Minnesota	S	B
3899	3900	52	Female	Handbag	Accessories	81	California	M	I

3900 rows × 18 columns



In [260...

```
df.isnull().sum()
```

Out[260...

```
Customer ID      0
Age              0
Gender           0
Item Purchased   0
Category         0
Purchase Amount (USD)  0
Location         0
Size            0
Color           0
Season           0
Review Rating    0
Subscription Status  0
Shipping Type    0
Discount Applied  0
Promo Code Used  0
Previous Purchases  0
Payment Method   0
Frequency of Purchases  0
dtype: int64
```

3. Group by Category

a) Group by product category to analyze sales and customer feedback

```
In [261...] category_summary = df.groupby('Category').agg(
    Total_Transactions=('Customer ID', 'count'),
    Total_Revenue=('Purchase Amount (USD)', 'sum'),
    Average_Spend=('Purchase Amount (USD)', 'mean'),
    Average_Rating=('Review Rating', 'mean')
).reset_index()

# Sort by Total Revenue in descending order
category_summary = category_summary.sort_values(by='Total_Revenue', ascending=False)

# Round decimals for cleaner presentation
category_summary['Average_Spend'] = category_summary['Average_Spend'].round(2)
category_summary['Average_Rating'] = category_summary['Average_Rating'].round(2)

df2
```

```
Out[261...]
   Category  Total_Transactions  Total_Revenue  Average_Spend  Average_Rating
1  Clothing                1737         104264           60.03           3.72
0  Accessories             1240          74200           59.84           3.77
2  Footwear                 599          36093           60.26           3.79
3  Outerwear                 324          18524           57.17           3.75
```

b) Group by Season and sort by Total Revenue descending

```
In [262...] season_summary = df.groupby('Season').agg(
    Total_Transactions=('Customer ID', 'count'),
    Total_Revenue=('Purchase Amount (USD)', 'sum'),
    Average_Spend=('Purchase Amount (USD)', 'mean'),
    Average_Rating=('Review Rating', 'mean')
).reset_index().sort_values(by='Total_Revenue', ascending=False)

season_summary['Average_Spend'] = season_summary['Average_Spend'].round(2)
season_summary['Average_Rating'] = season_summary['Average_Rating'].round(2)

season_summary
```

Out[262...

	Season	Total_Transactions	Total_Revenue	Average_Spend	Average_Rating
0	Fall	975	60018	61.56	3.73
1	Spring	999	58679	58.74	3.79
3	Winter	971	58607	60.36	3.75
2	Summer	955	55777	58.41	3.73

Fall Performance Leader: **Fall** generates the highest total revenue (60,018) and has the highest average spend per order (61.56), even though Spring actually records slightly more total transactions (999).

Consistent Distribution: **Transaction counts are extremely balanced across all four seasons** (ranging between 955 and 999 transactions), indicating steady year-round customer demand with a slight monetary bump during the Fall season.

c) Group by Payment Method and sort by Total Revenue descending

In [263...

```
payment_summary = df.groupby('Payment Method').agg(
    Total_Transactions=('Customer ID', 'count'),
    Total_Revenue=('Purchase Amount (USD)', 'sum'),
    Average_Spend=('Purchase Amount (USD)', 'mean')
).reset_index().sort_values(by='Total_Revenue', ascending=False)

payment_summary['Average_Spend'] = payment_summary['Average_Spend'].round(2)
payment_summary
```

Out[263...

	Payment Method	Total_Transactions	Total_Revenue	Average_Spend
2	Credit Card	671	40310	60.07
4	PayPal	677	40109	59.25
1	Cash	670	40002	59.70
3	Debit Card	636	38742	60.92
5	Venmo	634	37374	58.95
0	Bank Transfer	612	36544	59.71

Even Distribution: Payment choices are remarkably well-balanced across all methods. No single payment gateway completely dominates transaction volume or revenue.

Consistent Average Spend: The average order value per transaction remains tight across payment modes, ranging between 58.95 (Venmo) and 60.92 (Debit Card), indicating that payment method does not heavily skew basket size.

d) Group by both Category and Season, then sort by Total Revenue descending

```
In [264... multi_group = df.groupby(['Category', 'Season']).agg(
    Total_Transactions=('Customer ID', 'count'),
    Total_Revenue=('Purchase Amount (USD)', 'sum'),
    Average_Spend=('Purchase Amount (USD)', 'mean')
).reset_index().sort_values(by='Total_Revenue', ascending=False)

multi_group['Average_Spend'] = multi_group['Average_Spend'].round(2)
multi_group
```

```
Out[264... 
```

	Category	Season	Total_Transactions	Total_Revenue	Average_Spend
5	Clothing	Spring	454	27692	61.00
7	Clothing	Winter	448	27274	60.88
4	Clothing	Fall	427	26220	61.41
6	Clothing	Summer	408	23078	56.56
0	Accessories	Fall	324	19874	61.34
2	Accessories	Summer	312	19028	60.99
3	Accessories	Winter	303	18291	60.37
1	Accessories	Spring	301	17007	56.50
9	Footwear	Spring	163	9555	58.62
10	Footwear	Summer	160	9393	58.71
8	Footwear	Fall	136	8665	63.71
11	Footwear	Winter	140	8480	60.57
12	Outerwear	Fall	88	5259	59.76
15	Outerwear	Winter	80	4562	57.02
13	Outerwear	Spring	81	4425	54.63
14	Outerwear	Summer	75	4278	57.04

```
In [265... df.columns
```

```
Out[265... Index(['Customer ID', 'Age', 'Gender', 'Item Purchased', 'Category',
      'Purchase Amount (USD)', 'Location', 'Size', 'Color', 'Season',
      'Review Rating', 'Subscription Status', 'Shipping Type',
      'Discount Applied', 'Promo Code Used', 'Previous Purchases',
      'Payment Method', 'Frequency of Purchases'],
      dtype='str')
```

b) Creating a new column - Age Group

```
In [266... labels = ['Young Adult', 'Adult', 'Middle-aged', 'Senior']
df['Age Group'] = pd.qcut(df['Age'], q=4, labels = labels)
```

q=4: Tells pandas to split the *Age data* into 4 equal-sized buckets (quartiles: 0–25%, 25–50%, 50–75%, and 75–100% of the dataset). This ensures that roughly the same number of rows fall into each age group.

```
In [267... # Create Spending Tiers based on Purchase Amount

labels = ['Low', 'Medium', 'High']
df['Spending Tier'] = pd.qcut(df['Purchase Amount (USD)'], q=3, labels=labels)
```

```
In [268... df
```

Out[268...

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	
0	1	55	Male	Blouse	Clothing	53	Kentucky	L	
1	2	19	Male	Sweater	Clothing	64	Maine	L	Ma
2	3	50	Male	Jeans	Clothing	73	Massachusetts	S	Ma
3	4	21	Male	Sandals	Footwear	90	Rhode Island	M	Ma
4	5	45	Male	Blouse	Clothing	49	Oregon	M	Turq
...	...	...	...	...	...	...	...	...	
3895	3896	40	Female	Hoodie	Clothing	28	Virginia	L	Turq
3896	3897	52	Female	Backpack	Accessories	49	Iowa	L	V
3897	3898	46	Female	Belt	Accessories	33	New Jersey	L	C
3898	3899	44	Female	Shoes	Footwear	77	Minnesota	S	B
3899	3900	52	Female	Handbag	Accessories	81	California	M	I

3900 rows × 20 columns

c) Creating new column - 'Purchase Frequency Days'

```
In [269... frequency_mapping = {
    'Fortnightly': 14,
    'Weekly': 7,
    'Monthly': 30,
```

```

    'Quarterly': 90,
    'Bi-Weekly': 14,
    'Annually': 365,
    'Every 3 Months': 90
}

df['Purchase Frequency Days'] = df['Frequency of Purchases'].map(frequency_mapping)

```

```
In [270...] df[['Purchase Frequency Days', 'Frequency of Purchases']].head(10)
```

```
Out[270...]

```

	Purchase Frequency Days	Frequency of Purchases
0	14	Fortnightly
1	14	Fortnightly
2	7	Weekly
3	7	Weekly
4	365	Annually
5	7	Weekly
6	90	Quarterly
7	7	Weekly
8	365	Annually
9	90	Quarterly

```
In [271...] df.columns
```

```
Out[271...]
Index(['Customer ID', 'Age', 'Gender', 'Item Purchased', 'Category',
      'Purchase Amount (USD)', 'Location', 'Size', 'Color', 'Season',
      'Review Rating', 'Subscription Status', 'Shipping Type',
      'Discount Applied', 'Promo Code Used', 'Previous Purchases',
      'Payment Method', 'Frequency of Purchases', 'Age Group',
      'Spending Tier', 'Purchase Frequency Days'],
      dtype='str')
```

d) Checking all elements of two same column (identical)

```
In [272...] (df['Discount Applied'] == df['Promo Code Used']).all()
```

```
Out[272...] np.True_
```

```
In [273...]
# Dropping promo code used column

df = df.drop('Promo Code Used', axis=1)
```

```
In [274...] df.columns
```

```
Out[274...] Index(['Customer ID', 'Age', 'Gender', 'Item Purchased', 'Category',  
      'Purchase Amount (USD)', 'Location', 'Size', 'Color', 'Season',  
      'Review Rating', 'Subscription Status', 'Shipping Type',  
      'Discount Applied', 'Previous Purchases', 'Payment Method',  
      'Frequency of Purchases', 'Age Group', 'Spending Tier',  
      'Purchase Frequency Days'],  
      dtype='str')
```

```
In [275...] df.isnull().sum()
```

```
Out[275...] Customer ID          0  
Age          0  
Gender        0  
Item Purchased  0  
Category       0  
Purchase Amount (USD)  0  
Location       0  
Size          0  
Color         0  
Season        0  
Review Rating  0  
Subscription Status  0  
Shipping Type  0  
Discount Applied  0  
Previous Purchases  0  
Payment Method  0  
Frequency of Purchases  0  
Age Group      0  
Spending Tier  0  
Purchase Frequency Days  0  
dtype: int64
```

```
In [276...] df
```

Out[276...

	Customer ID	Age	Gender	Item Purchased	Category	Purchase Amount (USD)	Location	Size	
0	1	55	Male	Blouse	Clothing	53	Kentucky	L	
1	2	19	Male	Sweater	Clothing	64	Maine	L	Ma
2	3	50	Male	Jeans	Clothing	73	Massachusetts	S	Ma
3	4	21	Male	Sandals	Footwear	90	Rhode Island	M	Ma
4	5	45	Male	Blouse	Clothing	49	Oregon	M	Turq
...	...	...	...	...	...	...	...	...	
3895	3896	40	Female	Hoodie	Clothing	28	Virginia	L	Turq
3896	3897	52	Female	Backpack	Accessories	49	Iowa	L	V
3897	3898	46	Female	Belt	Accessories	33	New Jersey	L	C
3898	3899	44	Female	Shoes	Footwear	77	Minnesota	S	B
3899	3900	52	Female	Handbag	Accessories	81	California	M	I

3900 rows × 20 columns



In [277...

```
# Save the updated cleaned dataframe to a new CSV file
df.to_csv('updated_dataset.csv', index=False)
```

2: SQL Data Analysis

a) Simulating an SQL Database

In []:

```
import pandas as pd
import sqlite3
from sqlalchemy import create_engine

# =====
# SCENARIO A: Pulling Live Data from a Remote Database (e.g., PostgreSQL)
# =====
# Instead of pd.read_csv(), you use SQLAlchemy to connect to a live server string:
# format: 'postgresql://username:password@host:port/database_name'
```

```

engine = create_engine('postgresql://user:password@your-cloud-db.company.com:5432/r

# Pull live transactions dynamically into a pandas DataFrame
query = "SELECT * FROM transactions WHERE transaction_date >= CURRENT_DATE - INTERV
df = pd.read_sql(query, con=engine)

# =====
# SCENARIO B: Pulling Live Data from an Online API Endpoint
# =====
# import requests
# response = requests.get("https://api.company.com/v1/customer-behavior")
# df = pd.DataFrame(response.json())

# =====
# 2 & 3. Local Analytics Storage (SQLite)
# =====
# Once pulled online, you cache/store it locally in SQLite for fast Jupyter Lab que
conn = sqlite3.connect('shopping_behavior.db')
df.to_sql('customer_transactions', conn, index=False, if_exists='replace')

print(f"Successfully pulled {len(df)} live records and updated 'customer_transactio
conn.close()

```

In [278...

```

import sqlite3
import pandas as pd

# 1. Load the cleaned data
df = pd.read_csv('updated_dataset.csv')

# 2. Create an in-memory SQLite connection
conn = sqlite3.connect('shopping_behavior.db')

# 3. Write the DataFrame to an SQL table named 'customer_transactions'
df.to_sql('customer_transactions', conn, index=False, if_exists='replace')

print("Database and table 'customer_transactions' successfully created!")

```

Database and table 'customer_transactions' successfully created!

In [279...

df.columns

Out[279...

```

Index(['Customer ID', 'Age', 'Gender', 'Item Purchased', 'Category',
      'Purchase Amount (USD)', 'Location', 'Size', 'Color', 'Season',
      'Review Rating', 'Subscription Status', 'Shipping Type',
      'Discount Applied', 'Previous Purchases', 'Payment Method',
      'Frequency of Purchases', 'Age Group', 'Spending Tier',
      'Purchase Frequency Days'],
      dtype='str')

```

b) Query 1: Total Revenue and Average Spend by Product Category

Business Objective: Identify which product categories drive the highest revenue

In [280...

```
import pandas as pd
query_category = """
SELECT
    Category,
    COUNT("Customer ID") AS Total_Transactions,
    SUM("Purchase Amount (USD)") AS Total_Revenue,
    ROUND(AVG("Purchase Amount (USD)"), 2) AS Average_Spend,
    ROUND(AVG("Review Rating"), 2) AS Average_Rating
FROM customer_transactions

GROUP BY Category
ORDER BY Total_Revenue DESC;
"""

category_result = pd.read_sql(query_category, conn)
category_result
```

Out[280...

	Category	Total_Transactions	Total_Revenue	Average_Spend	Average_Rating
0	Clothing	1737	104264	60.03	3.72
1	Accessories	1240	74200	59.84	3.77
2	Footwear	599	36093	60.26	3.79
3	Outerwear	324	18524	57.17	3.75

Finding: **Clothing** and **Accessories** are the primary revenue drivers for the retail company, contributing significantly more sales volume than Footwear and Outerwear, while average purchase value remains consistent across all categories 57 to 60 dollar.

c) Query 2: High-Value Customers by Subscription Status

In [281...

```
query_segments = """
SELECT
    "Subscription Status",
    "Discount Applied",
    COUNT("Customer ID") AS Total_Customers,
    ROUND(AVG("Previous Purchases"), 1) AS Avg_Previous_Purchases,
    ROUND(AVG("Purchase Amount (USD)"), 2) AS Avg_Purchase_Amount
FROM customer_transactions

GROUP BY "Subscription Status", "Discount Applied"
ORDER BY Avg_Purchase_Amount DESC;
"""

segment_result = pd.read_sql(query_segments, conn)
segment_result
```

Out[281...

	Subscription Status	Discount Applied	Total_Customers	Avg_Previous_Purchases	Avg_Purchase_Amount
0	No	No	2223	25.1	60.13
1	Yes	Yes	1053	26.1	59.49
2	No	Yes	624	25.2	58.92

Consistent Spending: The average purchase amount remains remarkably stable (58.92 to 60.13) across all combinations, **showing that discounts and subscriptions do not heavily distort individual transaction sizes.**

Non-Subscriber Base: The **largest segment is non-subscribers who buy without discounts (2,223 customers)**, representing a massive target market for future loyalty program upsells.

d) Regional Performance: Top Locations by Revenue

Management wants to know which geographic locations yield the highest total sales and how ratings compare across regions.

In [282...

```
query_top_locations = """
SELECT
    Location,
    COUNT("Customer ID") AS Total_Transactions,
    SUM("Purchase Amount (USD)") AS Total_Revenue,
    ROUND(AVG("Purchase Amount (USD)"), 2) AS Avg_Spend,
    ROUND(AVG("Review Rating"), 2) AS Avg_Rating
FROM customer_transactions

GROUP BY Location
ORDER BY Total_Revenue DESC
LIMIT 5;
"""

top_locations_df = pd.read_sql(query_top_locations, conn)
top_locations_df
```

Out[282...

	Location	Total_Transactions	Total_Revenue	Avg_Spend	Avg_Rating
0	Montana	96	5784	60.25	3.82
1	Illinois	92	5617	61.05	3.64
2	California	95	5605	59.00	3.83
3	Idaho	93	5587	60.08	3.79
4	Nevada	87	5514	63.38	3.72

Montana leads all regions with the highest total revenue (5,784) and transaction volume (96), followed closely by Illinois and California.

Nevada stands out with the highest average spend per order (63.38) despite having fewer total transactions (87), indicating higher basket sizes in that region.

e) Multi-Column Grouping: Payment Method & Shipping Type Combinations

To optimize checkout experiences and logistics, management wants to see which payment preferences are most frequently paired with specific shipping methods.

```
In [283... query_payment_shipping = """
SELECT
    "Payment Method",
    "Shipping Type",
    COUNT("Customer ID") AS Transactions,
    SUM("Purchase Amount (USD)") AS Total_Revenue
FROM customer_transactions
GROUP BY "Payment Method", "Shipping Type"
ORDER BY Total_Revenue DESC
LIMIT 5;
"""

combo_df = pd.read_sql(query_payment_shipping, conn)
combo_df
```

Out[283...

	Payment Method	Shipping Type	Transactions	Total_Revenue
0	PayPal	Express	129	7695
1	Credit Card	2-Day Shipping	123	7411
2	PayPal	Free Shipping	123	7337
3	Cash	Free Shipping	121	7281
4	Debit Card	Free Shipping	117	7183

```
In [284... import sqlite3
import pandas as pd

conn = sqlite3.connect('shopping_behavior.db')

query_age_group = """
SELECT
    "Age Group",
    COUNT("Customer ID") AS Total_Customers,
    SUM("Purchase Amount (USD)") AS Total_Revenue,
    ROUND(AVG("Purchase Amount (USD)"), 2) AS Average_Spend
FROM customer_transactions
GROUP BY "Age Group"
ORDER BY Total_Revenue DESC;
```

```
"""
age_df = pd.read_sql(query_age_group, conn)
age_df
```

Out[284...

	Age Group	Total_Customers	Total_Revenue	Average_Spend
0	Young Adult	1028	62143	60.45
1	Middle-aged	986	59197	60.04
2	Adult	942	55978	59.42
3	Senior	944	55763	59.07

Finding: **Young Adult** and **Middle-aged** customers generate the highest share of total revenue 67,711 and 64,927 respectively), indicating that marketing campaigns should target these demographics for maximum profitability.

g) Using Spending Tier in SQL (Behavioral Value Segmentation)

Because we categorized spending into tiers (Low, Medium, High), you can filter or group by Spending Tier to see how subscription status or discounts correlate with spending level:

```
In [285... query_spending_tier = """
SELECT
    "Spending Tier",
    "Subscription Status",
    COUNT("Customer ID") AS Transaction_Count,
    SUM("Purchase Amount (USD)") AS Total_Revenue
FROM customer_transactions
GROUP BY "Spending Tier", "Subscription Status"
ORDER BY "Spending Tier", Total_Revenue DESC;
"""

spending_df = pd.read_sql(query_spending_tier, conn)
spending_df
```

Out[285...

	Spending Tier	Subscription Status	Transaction_Count	Total_Revenue
0	High	No	947	82794
1	High	Yes	350	30353
2	Low	No	947	30609
3	Low	Yes	353	11487
4	Medium	No	953	57033
5	Medium	Yes	350	20805

Highlights a major growth opportunity: introducing loyalty perks to convert **non-subscribers** (947 in the high tier) into recurring subscribers. Aso medium tier has count of

953 who have not subscribed.

h) Using Frequency of Purchases in SQL (Loyalty & Retention)

To find out how often customers return and which frequency category generates the most dependable revenue, query the frequency column directly:

In [286...

```
query_frequency = """
SELECT
    "Frequency of Purchases",
    "Purchase Frequency Days",
    COUNT("Customer ID") AS Total_Customers,
    ROUND(AVG("Previous Purchases"), 1) AS Avg_Previous_Purchases,
    SUM("Purchase Amount (USD)") AS Total_Revenue
FROM customer_transactions
GROUP BY "Frequency of Purchases"
ORDER BY Total_Revenue DESC;
"""

frq_df = pd.read_sql(query_frequency, conn)
frq_df
```

Out[286...

	Frequency of Purchases	Purchase Frequency Days	Total_Customers	Avg_Previous_Purchases	Total_Revenue
0	Every 3 Months	90	584	25.0	35088
1	Annually	365	572	24.6	34419
2	Quarterly	90	563	26.9	33771
3	Bi-Weekly	14	547	24.8	33200
4	Monthly	30	553	25.3	32810
5	Fortnightly	14	542	25.3	32007
6	Weekly	7	539	25.8	31786

In []:

In []:

In [287...

```
conn.close()
```

In []: